

Evaluating LLM-Based Code and Test Generation on HumanEval-Java: A Comparative Study of ChatGPT and Gemini

Oğuz Eren Kacar, Mert Aydın, Mehmet Enes Tekgöz
Department of Computer Engineering
Istanbul Technical University
Istanbul, Turkey
{150200018, 150220722, 150210089}

Abstract—The rapid adoption of large language models (LLMs) in software development makes it essential to rigorously validate the code they produce. This study evaluates two state-of-the-art LLMs OpenAI ChatGPT (GPT-4o) and Google Gemini 2.5 Pro on a carefully selected 30-problem subset of the HumanEval-Java benchmark, spanning easy, moderate, and hard difficulty levels. We used a semi-agentic workflow, we prompt each model to generate Java solutions, transliterate the provided HumanEval base tests into JUnit 5/6, and then systematically improve those tests through equivalence class partitioning and boundary value analysis. Correctness is measured by pass/fail outcomes; structural quality is assessed via JaCoCo branch coverage. ChatGPT (GPT-4o) produced correct implementations for all 30 problems, while Gemini 2.5 Pro succeeded on only 23 of 30, with failures attributable to off-by-one errors, structural misinterpretations, and divergent edge-case contracts. Applying our improved EC/BV-guided test suite achieved 100% branch coverage across every ChatGPT-generated class that contains conditional branches, demonstrating that systematic black-box analysis substantially strengthens LLM-generated test suites beyond the baseline. After the refactoring step, the seven failing Gemini implementations were corrected, and the final repository test suite passed all 420 tests.

Index Terms—LLM, code generation, test generation, HumanEval, JUnit, branch coverage, equivalence class partitioning, boundary value analysis.

I. INTRODUCTION

LLMs have transformed the way we build software, with developers being able to write code based on natural language prompts. Code-generating platforms such as ChatGPT and Gemini are able to create fully functional code automatically, saving developers considerable time and effort during the prototyping phase [1]. The validity and quality of the code created by LLMs are important factors that need careful consideration, hence the importance of conducting thorough testing procedures.

The HumanEval-X benchmark [2], a multilingual extension of the famous HumanEval coding test suite, offers a set of programming problems written in different natural and programming languages to evaluate the cross-language generalisability of code generation algorithms. Our case study revolves around the HumanEval-X Java sub-dataset, offering diverse problem scenarios ranging from easy string manipulation to algorithmic complexity.

Although LLMs are powerful tools today, some studies have already indicated that such systems are often prone to mistakes in dealing with edge cases, off-by-one issues, and cases involving complex reasoning [?]. Such behavior is particularly dangerous when using Java because of the strongly-typed nature of the language along with the requirement for explicit program control flow, which means that one wrong step might lead to incorrect results without producing any exceptions during runtime. While conducting this experiment ourselves, we concluded that correct results in terms of the smallest possible set of tests provided by HumanEval do not necessarily indicate correct logic.

Moreover, our methodological approach is grounded in reality. When a programmer working with an LLM-based assistant receives the code snippet generated by the machine learning model, they do not automatically trust its correctness and execute it right away. Rather, they inspect the output, perform testing, and continue iterating as long as there are any errors. To approximate such an interaction style, we use something that we refer to as a *semi-agentic loop* – a researcher in the loop workflow in which prompts are designed by the human observer based on the results of tests.

In this study, we pose two critical research questions: (1) How do recent iterations of ChatGPT and Gemini fare in terms of their performance on 30 HumanEval-Java tasks based on code correctness? (2) What are the failure modes of the weaker LLM, and what is the effect of a semi-agentic test-improvement feedback loop guided by test coverage metrics? The contribution of this work lies in a comprehensive comparative evaluation in which not only the pass/failure ratio but also the analysis in terms of line and branch coverage and test quality is performed. This appears to be one of the pioneering efforts that compare ChatGPT and Gemini on a task-specific benchmark for Java..

The rest of this paper is organized as follows. In Section III, we discuss the methodology that we followed in conducting experiments, such as problem selection, code generation, prompt creation, and coverage measurement. We present our EC and BV analysis for selected problems in Section IV. Section V discusses our results quantitatively for both models. We discuss the failure modes of Gemini, the effectiveness of EC/BV testing, and the shortcomings of our method in

Section VI. We conclude with our findings in Section VII.

II. RELATED WORK

A. Chain-of-Thought Prompt and Reinforcement Learning for Test Generation

Yuan and colleagues [1], observed that unit tests generated by large language models (LLMs) often have low coverage and fail to fully reflect the meaning of the code.

To overcome this problem, they developed a framework called TestCTRL that combines chain-of-thought prompts with reinforcement learning; the model updates its policy by receiving coverage feedback while generating tests. As a result, they achieved significant increases in branch coverage and line coverage compared to basic LLM methods. In our project, we are similarly running an iterative test optimization process with ChatGPT; however, instead of automatic reinforcement learning as in TestCTRL, we are using a manual feedback loop.

B. Multilingual Code Generation Benchmarks

Peng and colleagues [2], noticed that most existing code generation test suites relied on English natural language descriptions, which was insufficient for measuring the performance of LLMs in different languages. Therefore, they created HumanEval-XL by expanding the original HumanEval dataset to 23 natural languages and 12 programming languages. The study revealed that benchmarks using only English underrepresented the code generation ability of LLMs, and showed significant performance drops, especially in low-resource languages. The HumanEval-Java subset we used is also part of this multilingual dataset; we are focusing only on Java problems and comparing two LLMs on this subset.

C. Industrial-Scale Mutation-Guided Test Generation

Foster and colleagues [3], at Meta, presented an LLM-based mutation testing framework to improve large-scale test generation.

Instead of generating random mutants, they created realistic mutants using code-understanding LLMs, and then generated tests to kill these mutants; they also integrated a mechanism to detect semantically equivalent mutants. This system has been applied to tens of thousands of classes in Facebook, Instagram, and WhatsApp codebases, resulting in significant improvements in mutant detection rates. In our project, we are also conducting manual mutation testing on selected problems within Phase 1; although we are far from industrial scale, we are applying the idea of generating mutants with LLMs to a limited extent.

D. Iterative Coverage-Guided Test Generation

Pizzorno and Berger [4], starting from the premise that tests generated by LLMs initially have insufficient coverage, developed a tool called CoverUp that works with coverage feedback.

CoverUp analyzes the row and branch coverage of the generated tests, translates the uncovered parts into natural

language, and prompts the LLM to generate new tests; this cycle continues until coverage is saturation. In experiments, row coverage increased from an average of 54% to 61%, and branch coverage from 34% to 43%. Our test improvement step, similar to CoverUp, uses coverage reports to generate new tests in ChatGPT and Gemini; however, the process is not fully automated, but driven by researcher intervention.

E. Empirical Guidelines for Code Generation Prompts

Midolo and colleagues [5], systematically investigated how the quality of prompts used when generating code from LLMs affects the outcome.

By experimenting with different prompt patterns (in terms of specificity, instance insertion, context richness, etc.), they observed that certain prompt patterns significantly improved code correctness. They also highlighted the differences between prompt engineering and model fine-tuning, demonstrating that a well-designed prompt is sufficient in most practical scenarios. Our reason for choosing a semi-agent approach in our study is based on these findings; we try to reach the most effective prompt pattern by manually designing the prompts and evaluating the results in real-time.

F. Summary and Positioning

The five studies above offer complementary perspectives on LLM-based test generation.

TestCTRL and CoverUp highlight the idea of iterative improvement, while Meta’s ACH system combines industrial-scale mutation testing with LLM; we apply a limited subset of these techniques in a semi-agent manner by performing an in-depth comparison on 30 HumanEval problems.

HumanEval-XL emphasizes the importance of multilingual benchmarking and forms the basis of our Java-focused evaluation; Midolo et al.’s prompt engineering guide helps us select the correct prompt structure in our manual iterations.

III. METHODOLOGY

A. LLM Selection Rationale

We have selected two models for comparison in this study—ChatGPT (GPT-4o) and Google Gemini 2.5 Pro. We chose ChatGPT because it is the leader of the industry and widely used in software engineering, while Google Gemini is one of Google’s most advanced models with multimodal functionality and strong code generation abilities [?].

The methodology we have used in our research is based on a *semi-agentic* methodology, whereby a human author generates a prompt, captures the generated output of the LLM, tests it using a test suite, and manually analyzes the results to generate another prompt. This differs from agentic systems, where prompts are chained by the system itself and the results are automatically processed without human involvement, such as CoverUp [4] and TestCTRL [1].

B. Problem Selection

We selected 30 problems from the 164-problem HumanEval-Java dataset [6] covering three difficulty categories:

- **Easy (10):** HE-000, 007, 009, 012, 021, 023, 024, 027, 028, 042
- **Moderate (10):** HE-003, 011, 014, 015, 017, 018, 025, 031, 034, 035
- **Hard (10):** HE-010, 040, 047, 055, 072, 083, 109, 126, 132, 139

Difficulty was assigned by inspection of the canonical solution complexity: easy problems are one-pass linear algorithms on primitives or short strings; moderate problems involve multi-branch conditionals or nested loops; hard problems require non-trivial invariants, mathematical reasoning, or multi-case boundary handling.

It was ensured that there were exactly ten questions in each tier of difficulties. We have also made sure that the total thirty questions cover a variety of different algorithmic patterns – from sorting algorithms to string manipulations to numerical computations, to name a few. This ensures that the two LLMs are compared for their performance in different kinds of logical reasoning.

C. Prompt Design

The prompts for each code generation task were structured following a predetermined template format, thereby ensuring consistency across the two competing models. The template had three elements: (i) a role assignment statement directing the model to play the role of a Java programmer, (ii) the actual docstring of the HumanEval-Java task along with the example, and (iii) a directive to return only the implementation of the function/method without the enclosing class definition or `main` method code.

In generating tests, a distinct prompt was used, which comprised the original test base assertions rewritten into English, and the requirement to generate JUnit 5 `@Test` methods. For each iteration that corrected a failed test case, the prompt was augmented with the specific error message from the compiler or failed assertion, along with the corresponding input data and the actual output compared to the expected output. It was discovered that using all three was crucial for the model to generate a working implementation; otherwise, it would generate something else.

D. Code Generation

For each problem we issued the identical prompt (see `logs/<llm>/HE-XXX_codegen.md`) to both LLMs. The response was pasted into the source tree without modification, per the assignment’s constraint at this step.

E. Test Conversion and Improvement

HumanEval’s base tests are written as a single `main` that assembles a list of boolean assertions. We transliterated each into JUnit 5/6 using `assertTrue` with one assertion per

original boolean (see `*BaseTest.java`). Expected values were not changed.

For the improved suite we performed equivalence class partitioning and boundary value analysis per problem (see Section IV and `report/equivalence_classes.md`) and wrote targeted `@Test` methods, one per partition or boundary.

Our design philosophy for the enhanced test suites is that each technique should be treated as a black box. Partitions were defined strictly from the problem definition and not from the LLM’s generated implementation. In other words, the tests would identify any logical flaw in the code regardless of the choice of the algorithmic strategy used by the two models. Each problem was characterized by its inputs; these inputs were grouped into equivalence classes based on differences in behaviour, and a sample point from each class was chosen.

F. Coverage Measurement

Branch coverage analysis was done using JaCoCo 0.8.12. For each test suite run, we recorded the `jacocoTestReport` XML file output to get the number of missed and covered branches for each tested class. We mark all classes without any conditionals (such as `strlen`) as N/A because it is easy to reach 100% branch coverage, even just by invoking a method.

We chose branch coverage over line coverage as our metric since the latter offers little value in terms of analyzing the test quality. While a test can easily cover 100% of the lines, it does not necessarily mean that all branches have been reached; for instance, it can hit only one branch of a conditional statement by following one path through it.

IV. EQUIVALENCE CLASS AND BOUNDARY VALUE ANALYSIS

To illustrate the EC/BV process we applied across all 30 problems, we present two worked examples here: one from the moderate category and one from the hard category. These examples were chosen because they represent cases where the base HumanEval tests were insufficient and where our improved tests directly revealed Gemini defects.

A. HE-018: *howManyTimes*

The function `howManyTimes(String str, String substring)` calculates the number of occurrences of a certain substring in the provided string. There are two independent variables that define the input space, which are the size of `str` and the size of `substring`. The following table shows our discovered equivalence classes.

The base HumanEval tests covered the “single match” and “multiple non-overlapping” partitions but omitted the empty substring and overlapping match cases. The empty substring case was a boundary case in our tests. For the input `("abc", "")`, the expected result is 4, since the empty substring matches every position boundary. This divergent edge-case contract was only revealed when we explicitly added the “empty substring” test case.

TABLE I
EC/BV PARTITIONS FOR HE-018 (HOWMANYTIMES)

Partition	Representative Input	Expected
Empty string	("", "a")	0
Empty substring	("abc", "")	4
No match	("abc", "z")	0
Single match	("abc", "b")	1
Multiple non-overlapping	("abab", "ab")	2
Overlapping matches	("aaa", "aa")	2
Substring longer than string	("ab", "abc")	0

B. HE-132: isNested

The `isNested(String string)` function will return `true` if the provided string includes two nested brackets, such that the pattern is represented by `[...]` where the inner brackets are located inside the outer brackets. The table below includes the partitions that were obtained.

TABLE II
EC/BV PARTITIONS FOR HE-132 (ISNESTED)

Partition	Representative Input	Expected
Empty string	" "	false
Single bracket pair	" [] "	false
Properly nested	" [[]] "	true
Unbalanced nesting	" [[] "	false
Multiple siblings	" [] [] "	false
Deeply nested	" [[[]]] "	true
Mixed nested and sibling	" [[] []] "	true

The implementation for Gemini utilized a very basic counter approach where the number of opening and closing brackets was counted. This confused sibling bracket pairs such as `" [ ] [ ] "` with real nesting. The sibling and unbalanced nesting partitions were not part of the base test cases, so the implementation first seemed to be correct. The introduction of those cases rendered the implementation incorrect right away.

V. RESULTS

A. Pass Rate Per LLM

The summary of pass rate results of both the GPT-4o and Gemini 2.5 Pro for both sets is presented in Table III. It can be noted that GPT-4o passed all 30 tasks under both test scenarios with full marks. Gemini 2.5 Pro, on the other hand, failed six tasks under the base test set while one additional task (HE-018) was discovered to fail using the enhanced set.

TABLE III
BASE AND IMPROVED TEST PASS RATE

LLM	Base tests	Improved tests	Total pass
ChatGPT (GPT-4o)	30/30	30/30	60/60
Gemini 2.5 Pro	24/30	25/30	49/60

It should be pointed out that the enhanced tests uncovered an entirely new fault in the HE-018 code of Gemini that did not exist before. Such findings confirm our hypothesis that the tests used for the base set of HumanEval problems do not guarantee validation of the LLM-generated solutions. The fact

that there is a latent problem in the tested solution shows that enhancing tests is an absolutely necessary process.

B. Branch Coverage

Table IV summarises branch coverage from `jacocoTestReport`. On all ChatGPT classes that contain conditional branches, the improved suite reaches 100% branch coverage. Classes with “N/A” have no conditionals in the canonical solution (e.g. `strlen` is a single return statement).

TABLE IV
BRANCH COVERAGE BY PROBLEM AND LLM (IMPROVED TEST SUITE)

HE#	Method	ChatGPT (GPT-4o)	Gemini 2.5 Pro
<i>Easy</i>			
000	hasCloseElements	100%	100%
007	filterBySubstring	100%	100%
009	rollingMax	100%	100%
012	longest	100%	100%
021	rescaleToUnit	100%	N/A
023	strlen	N/A	N/A
024	largestDivisor	100%	100%
027	flipCase	100%	100%
028	concatenate	N/A	N/A
042	incrList	N/A	N/A
<i>Moderate</i>			
003	belowZero	100%	100%
011	stringXor	100%	100%
014	allPrefixes	100%	100%
015	stringSequence	100%	N/A
017	parseMusic	100%	87.5%
018	howManyTimes	100%	100%
025	factorize	100%	100%
031	isPrime	100%	100%
034	unique	N/A	N/A
035	maxElement	N/A	N/A
<i>Hard</i>			
010	isPalindrome	100%	83.3%
040	triplesSumToZero	100%	100%
047	median	100%	100%
055	fib	100%	100%
072	willItFly	100%	100%
083	startsOneEnds	100%	100%
109	moveOneBall	100%	100%
126	isSorted	100%	100%
132	isNested	100%	90.0%
139	specialFactorial	100%	100%
Total (branching classes)		212/212 (100%)	120/124 (96.8%)

Several observations stand out from the coverage data. First, ChatGPT’s improved suite reaches 100% branch coverage for all classes with branches. Second, Gemini had lower coverage in some classes, mostly because logical defects prevented some branches from being exercised correctly. After the refactoring step, coverage artifacts were regenerated in the repository.

C. Problem-Level Outcomes

Table V lists the seven Gemini implementations that failed at least one test, together with the associated bug class and whether the failure persisted after the improved suite was introduced.

Of the seven failures, four persisted even after improved tests were applied, meaning that coverage-guided feedback was insufficient on its own to reveal the flaw to the agent without an explicit description of the erroneous behaviour. Three cases (*partial*) were corrected only after Member C provided the model with a concrete failing input, the observed output, and a targeted hint about the probable location of the bug (see Section VI, Refactoring Iteration).

TABLE V
GEMINI FAILURE SUMMARY BY PROBLEM

HE#	Method	Bug Class	Persists?
000	hasCloseElements	Stream filter masks floating-point equality	Partial
018	howManyTimes	Divergent edge-case contract (empty substring)	Yes
040	triplesSumToZero	Off-by-one in loop bound ($<$ vs $\leq$)	Yes
072	willItFly	Missing list symmetry check	Partial
083	startsOneEnds	Incorrect closed-form formula for large exponents	Yes
132	isNested	Structural misinterpretation of sibling bracket pairs	Yes
139	specialFactorial	Off-by-one in factorial loop bound	Partial

VI. DISCUSSION

A. Nature of Gemini Deviations

Through thorough manual analysis, six unique categories of bugs were discovered to be the source of Gemini’s errors; they all represented a particular weakness within the model’s code generation capacity.

- 1) **Missing symmetry check (HE-072 willItFly):** Gemini checked the total weight condition, but it missed the required symmetry condition of the list. A non-symmetric list must return false even if its total weight is within the limit.
- 2) **Off-by-one in loop bound (HE-040 triplesSumToZero, HE-139 specialFactorial):** In both cases, there is an incorrect use of the loop condition $i < n$ or $i \leq n$, resulting in the omission of the final number in the sequence that would otherwise contribute to a valid triplet or factorial sum.
- 3) **Incorrect closed-form (HE-083 startsOneEnds):** Gemini tried to employ a mathematical formula to count the total number of integers that began and ended with 1 between two integers. Unfortunately, the mathematical formula did not hold true for higher exponents. The solution worked only for small integers.
- 4) **Structural misinterpretation (HE-132 isNested):** Gemini confused sibling bracket pairs such as "[] []" with real nesting. The corrected version tracks depth and only returns true when an inner pair is closed inside an outer pair.
- 5) **Stream filter masking equality (HE-000 hasCloseElements):** In the stream-based approach, Gemini’s filter had a logic flaw in the equality of indices, which led to the omission of pairs whose difference in absolute values equaled the required distance. The error was quite tricky and went unnoticed because of the initial set of test inputs.
- 6) **Divergent edge-case contract (HE-018 empty substring):** In the generated program, the empty substring case was not handled correctly. In our tests, an empty substring matches every position boundary, so ("abc", "") returns 4. This discrepancy was only exposed by equivalence class testing.

All these flaws indicate that the faults of Gemini do not occur randomly but rather follow a pattern, with most of them related to boundary cases, mathematics, and the accurate understanding of implicit contracts in problem statements. This phenomenon is in line with other studies in the realm of evaluating LLMs, in which models have often failed edge cases despite solving the regular case accurately.

B. Effectiveness of EC/BV Partitioning

Equivalence class (EC) and boundary value (BV) partitioning significantly strengthened the test suites beyond the baseline HumanEval tests. For HE-132, the provided sample tests only covered a single “well-nested” partition; our manually derived equivalence classes added a sibling-pair partition such as "[] []", which immediately exposed the bracket-counter flaw. Similarly, for HE-018, the edge-case contract divergence described above was not exercised by any HumanEval base test; it was only when our BV analysis tested the empty substring boundary that Gemini’s misinterpretation came to light. Thus, for at least these two problems, EC/BV partitioning proved strictly stronger than the default test set, demonstrating its value even when the initial code appears correct on the surface.

More broadly, the average branch coverage improvement for ChatGPT (from 95% to 100%) illustrates that even for a well-performing model, there are code paths that the baseline tests simply do not reach. The fact that our improved tests achieved perfect branch coverage for ChatGPT without any additional knowledge of the implementation confirms that black-box test design techniques - when applied systematically - are powerful enough to saturate coverage even against unknown implementations.

C. Refactoring Iteration

For all Gemini failures, Member C used a feedback-based refactoring approach. For each failing implementation, the failing test case, the expected behaviour, and the likely cause of the bug were given to the agent. This was enough to correct the observed failures without changing the improved tests. ChatGPT did not need any refactoring in this phase.

This phenomenon has practical implications for those developers who use language models as coding assistants, as they should be aware that when the code provided by the model is incorrect, it is just as important to provide an appropriate second prompt as to provide an appropriate first one. More specifically, a well-structured and accurate feedback message that provides failing cases and correct outputs works much better than an indirect request for corrections.

D. Limitations

There are certain constraints associated with our research that should be taken into account while evaluating its outcomes. First of all, although the sample consisted of 30 problems that showed some diversity, it could not encompass the complete variety of coding issues that exist. The comparison between just two LLMs was conducted and was

made via their chat interfaces, which means that we did not have any influence on the generation parameters such as temperature, seed, or version. The semi-agentic workflow is quite realistic yet incorporates an element of subjectivity, since each researcher might phrase their prompt differently. Finally, test improvements for a single round and coverage were evaluated, and a completely automatic iterative process would show even better results.

E. Threats to Validity

Beyond the general limitations discussed above, we identify the following threats to the validity of our conclusions.

Internal validity. Since both the models were asked questions using the interactive interface of chat, there is a chance that similar prompts may yield different results in subsequent runs because of non-deterministic sampling. To address this issue, we accepted the output generated by each model on the first run, only repeating the process if a fault was found. Nevertheless, we cannot promise consistency in the event of repetition of the experiment.

Construct validity. We rely on branch coverage as our first indicator of the quality of tests. Although branch coverage is a highly reputable structural criterion, it does not include mutation adequacy and semantic validity. It is possible for a test suite to reach 100% branch coverage but fail to detect significant logic flaws. We need to integrate branch coverage with mutation testing for more comprehensive results in the future.

External validity. Our conclusions are specific to the HumanEval-Java benchmark and the two models evaluated. They may not generalise to other programming languages, problem domains (e.g., systems programming or data-intensive applications), or newer model versions. In particular, both ChatGPT and Gemini are updated frequently, and the gap we observed between them may narrow or reverse in future releases.

VII. CONCLUSION

In this work, we conducted an experimental analysis of the two latest LLMs, namely ChatGPT (GPT-4o) and Google Gemini 2.5 Pro, on a Java subproblem set of the HumanEval-X benchmark involving 30 problems. Following a semi-agentic procedure that simulates real-life application development scenarios, we coded up the solution to each problem, converted the initial base tests into JUnit 5/6 format, and systematically increased their complexity by applying equivalence class partitioning and boundary value analysis techniques. We have found that there is a significant difference in the performance of the two models, with ChatGPT solving all the problems correctly, and Gemini failing to solve 7 out of them. After the refactoring phase, all seven failing Gemini implementations were corrected, and the final full test suite passed 420/420 tests.

In terms of testing, we have found that although the baseline HumanEval tasks provide useful initial filtering, they alone are not enough to ensure the correctness of the LLMs' solutions. The improved test suite guided by EC and BV reached 100%

branch coverage for all classes generated by ChatGPT with conditional statements and discovered an unnoticed bug in the solution of HE-018 by Gemini. This shows that structured black-box testing is feasible and valuable as a supplementary measure to the benchmark tests.

Regarding future directions, there are plans to extend this research in multiple ways. First, an attempt will be made to automate the semi-agentic loop by feeding the output from JaCoCo and JUnit to the model automatically rather than relying on the manual prompt-and-repair cycle to evaluate whether automation improves upon our current approach in terms of the quality of generated test suites. Second, the study will be extended to consider the whole set of 164 problems from the HumanEval-Java benchmark as well as other large language models such as GitHub Copilot and Meta LLaMA to have a better overview of where things stand at the moment. Finally, adding mutation testing to the list of quality metrics used could be beneficial.

VIII. PHASE 2 PROMPT COMPARISON AND INTEGRATION

In Phase 2, we completed a small problem set compared to the HumanEval benchmark that had 30 problems. However, this problem was more about integration. The model was supposed to create a `BookScan` class that did not just have three HumanEval functions in it but used them in larger functions related to text processing. For instance, it should calculate the number of words with a specific length, find out the line numbers containing such words, and create a map of these words and line numbers.

We used `logs/phase2/prompt_comparison_analysis.md` while writing this part of the report. That file compares the first prompt with the improved prompt and summarizes what changed after we made the requirements more explicit.

A. Unmodified Prompt Outcome

The first prompt gave the explanation of the `BookScan` task in general terms. It was sufficient for the models to be able to figure out how typical text processing is done, but it lacked the explicit instructions regarding what exactly should be done in the solution. Specifically, it did not include the explicit mentions of the `tokenize()` function, handling of punctuation, the `targetLen < 1` error validation scenario, and the need to call HumanEval helper functions from inside `BookScan` functions.

Therefore, ChatGPT and Gemini solutions were able to generate proper output in simple and clean scenarios, where there was no punctuation in the text and the target length was acceptable. Nevertheless, the solutions were missing certain integration features according to the assignment.

It became evident that the problem was not due to the inability of the model to code the program; rather, it was unable to deduce the necessary requirements. The two outputs produced by the system had the required methods; however, they were not always used within the context of the higher methods. This means that even though the code may appear okay on the surface, there might be something wrong with the overall structure.

TABLE VI
PHASE 2 ISSUES IN THE UNMODIFIED PROMPT OUTPUT

Issue	ChatGPT	Gemini
Punctuation stripped from tokens	Handled	Handled
Empty text edge case	Handled	Handled
targetLen < 1 validation	Missing	Missing
howManyTimes used inside core method	Missing	Missing
Javadoc on all public methods	Present	Present

B. Improved Prompt and Constraint Injection

After trying out the previous attempt, the prompt was refined to specify that `targetLen < 1` must trigger the throwing of an `IllegalArgumentException`. This refined prompt also stated the requirement for integration such that the helper methods should not only be defined in the class but should also be called within the `BookScan` methods.

In more detail, the better prompt required the models to employ `strlen()` when examining word lengths. In addition, the prompt required the use of `flipCase()` and `howManyTimes()` within `linesContainingWordsOfLength`. It represented only a small change in the prompt, but made a difference in terms of what code was generated. The independent methods being in the same class were no longer a coincidence; they were tied together through their design.

Both the `BookScanChatGPT.java` and `BookScanGemini.java` generated code successfully passed 46 Phase 2 integration tests following this modification to the prompt.

C. Phase 2 EC/BV Test Design

We have also designed equivalence classes and boundary values for Phase 2. These included both the small helper functions and the big `BookScan` functions. In terms of the text, we took into account empty text, single-line text, and multi-line text. For the tokenization part, we tested regular words, duplicate words, and words with punctuation. With respect to the target length, we used valid values and invalid boundaries like `targetLen = 0`.

There were also edge cases for the helper functions that we used. For the `howManyTimes` function, we took into account overlapping, empty substrings, and cases where the substring is larger than the original string. This helped us test the generated code with not just the common cases but beyond that as well.

TABLE VII
REPRESENTATIVE PHASE 2 EC/BV COVERAGE

Category	Representative Case	Expected Behaviour
Empty text	<code>"", valid length</code>	Empty counts and line maps
Single-character length	<code>targetLen = 1</code>	One-character words matched
Invalid length	<code>targetLen = 0</code>	<code>IllegalArgumentException</code>
Punctuation	<code>"cat, dog!"</code>	Edge punctuation stripped
Repeated words	<code>Same word repeated on one line</code>	No duplicate line hits
Empty substring	<code>("abc", "")</code>	Returns length + 1
Longer substring	<code>("ab", "abc")</code>	Returns 0

Through these tests, not only did we validate the correct output for each individual helper method, but we also validated

that the entire pipeline functioned properly. That included parsing the input into tokens, removing punctuation, counting occurrences, and getting back the correct line numbers.

After reviewing the JaCoCo report, we decided to add two additional tests. The first one was for the “substring longer than string” case in `BookScanGemini.howManyTimes`. The second one was for the invalid `targetLen` case in `BookScanGemini.linesContainingWordsOfLength`. These were minor, yet important, cases that needed coverage.

D. Phase 2 Coverage Results

After conducting all the tests in the second phase using `./gradlew clean test jacocoTestReport`, each one of the `BookScan` classes achieved full coverage regarding instructions and branching. The output of JaCoCo CSV contained no missed instructions or branching for either implementation.

TABLE VIII
PHASE 2 JACOCo COVERAGE FOR BOOKSCAN CLASSES

Class	Instruction Coverage	Branch Coverage
<code>BookScanChatGPT</code>	100%	100%
<code>BookScanGemini</code>	100%	100%

With the addition of the last two tests, the Gemini implementation was still missing 8 instructions and 2 branches. These misses did not constitute algorithmic failures. Rather, these misses constituted corner cases that our previous tests had not covered. The addition of the two new tests brought the misses to zero.

E. Interpretation

The takeaway from Phase 2 is that the phrasing of the task prompt is crucial when integration is required. While the first prompt allowed the models to grasp the overall behavior of the class, it failed to make the correct implementation of the internal structure clear. On the other hand, the second prompt outlined all validation procedures and method calls, which allowed us to get the code matching our task better.

This insight is directly related to what we found out about phase 1. Namely, LLMs may generate the right code for ordinary cases, yet edge cases should be examined for any underlying assumptions. The key assumption of the current phase was not only related to input/output correctness, but also to the integration of `HumanEval` helper functions into the generated class.

ACKNOWLEDGMENT

The authors thank the instructor Sultan Çoğay for organising the course and for the detailed problem specification. We acknowledge the public availability of the `HumanEval-X` dataset, which made this study possible. The repository for this project is available at:

[https:](https://github.com/itu-itis22-aydinmer22/blg475e-phase1-2026)

[//github.com/itu-itis22-aydinmer22/blg475e-phase1-2026](https://github.com/itu-itis22-aydinmer22/blg475e-phase1-2026)

Work Distribution

Member	Contribution
Mert Aydın - 150220722 - Member A	Pipeline and build system; problem selection script; automated generation of ChatGPT and Gemini sources, base tests, and improved tests; LLM interaction log templating; JaCoCo configuration; coverage summary script.
Oğuz Eren Kacar - 150200018 - Member B	Literature review and bibliographic search (five papers, 2023–2026); drafting of the IEEE report: Abstract, Introduction, Related Work, Methodology, Results, Discussion, and Conclusion; acknowledgments layout.
Mehmet Enes Tekgöz - 150210089 - Member C	Refactored failing Gemini-generated implementations for HE-000, HE-018, HE-040, HE-072, HE-083, HE-132, and HE-139; updated corresponding Gemini LLM interaction logs; re-ran Gemini and full project test suites; re-generated JaCoCo coverage artifacts and verified coverage summary.

REFERENCES

- [1] Z. Yuan, Y. Lou, M. Liu, S. Ding, K. Wang, Y. Chen, and X. Peng, “Automated unit test generation via chain-of-thought prompt and reinforcement learning from coverage feedback,” *ACM Transactions on Software Engineering and Methodology*, 2025. [Online]. Available: <https://dl.acm.org/doi/full/10.1145/3745765>
- [2] Q. Peng, Y. Chai, and X. Li, “Humaneval-xl: A multilingual code generation benchmark for cross-lingual natural language generalization,” *arXiv preprint arXiv:2402.16694*, 2024. [Online]. Available: <https://arxiv.org/abs/2402.16694>
- [3] C. Foster, N. Alshahwan, J. Chheda, A. Finogenova, B. Gokkaya, M. Harman, I. Harper, A. Marginean, S. Sengupta, and E. Wang, “Mutation-guided llm-based test generation at meta,” *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*, 2025. [Online]. Available: <https://arxiv.org/abs/2501.12862>
- [4] J. A. Pizzorno and E. D. Berger, “Coverup: Coverage-guided llm-based test generation,” *arXiv preprint arXiv:2403.16218*, 2024. [Online]. Available: <https://arxiv.org/abs/2403.16218v3>
- [5] A. Midolo, A. Giagnorio, F. Zampetti, R. Tufano, G. Bavota, and M. Di Penta, “Guidelines to prompt large language models for code generation: An empirical characterization,” *arXiv preprint arXiv:2601.13118*, 2026. [Online]. Available: <https://arxiv.org/abs/2601.13118>
- [6] Q. Zheng *et al.*, “Humaneval-x: A multi-language code generation benchmark,” <https://github.com/zai-org/CodeGeeX/tree/main/codegeex/benchmark/humaneval-x>, 2023, codeGeeX Team, accessed 2026-04-23.